1. Introduction

We study a systems architecture for using learned models inside checked search. Three components work together: a verifier-grounded ranking loop that separates validity from preference, a holographic memory substrate that stores certified rules in fixed-dimension vector space, and an EML-augmented neural scorer that improves ranking accuracy through a universal continuous primitive.

The core loop is simple. Enumerate candidate actions. A symbolic gate rejects provably invalid candidates. A small neural model ranks the survivors. A deterministic verifier confirms or refutes the top choice. The trace is preserved, and empirical claims are bound to the generated evidence. Nothing about this pipeline is new individually, but the integration produces a discipline: the model never owns validity, only ranking pressure, and every claim in the paper is mechanically checked against the repository's output artifacts.

We make three contributions:

Strong verified anchor. On arithmetic sequence filtering (next-prime gap prediction with modular divisibility gates), a learned rule grammar discovers rejection predicates that eliminate impossible candidates with zero false rejections. The learned wheel achieves 100% top-1 accuracy on the declared support, improving on the hand-coded wheel by +0.02 pp percentage points while remaining verifier-certified.

Holographic rule memory. Certified rules are stored as vector-symbolic bindings on the unit hypersphere, enabling content-addressable retrieval in fixed O(d) memory regardless of how many rules are stored. The holographic ranker achieves retrieval fidelity proportional to 1/sqrt(N) with zero trainable parameters, competing with gradient-trained neural scorers on small sample counts.

EML neural scoring. We introduce the EML operator, eml(x,y) = exp(x) - ln(|y|+epsilon), as a universal primitive for continuous functions. Integrated into the scorer as pairwise feature enrichment, a smooth activation function, and a stackable pairwise layer with learned weights, EML improves accuracy on nonlinear benchmarks from 92.2% to 94.8% at identical width, and stacked EML layers compose correctly at 94.5% accuracy.

The rest of the paper is organized around the data flow. Section 2 defines the verifier-grounded protocol. Section 3 presents the arithmetic filtering anchor. Section 4 describes the holographic memory substrate. Section 5 presents the EML scorer architecture. Section 6 covers claim governance. Section 7 discusses limitations and boundary evidence.

2. Protocol

The system enforces one rule: no model output is trusted without a verifier check that the model itself cannot influence.

2.1. The Verified Search Loop

Every decision follows the same five-step protocol:

1. **Enumerate.** Given a state, generate the full candidate action set. No model-based pruning at this stage.
2. **Gate.** Apply symbolic rules with machine-checked certificates. A rule is accepted only if its certificate proves it never rejects a valid action within the declared support. Invalid candidates are removed.
3. **Rank.** A neural scorer assigns a score to each surviving candidate. The model sees only features explicitly tagged by provenance (state observation, structural prior, or verifier-derived). Leakage from the verifier into the scorer's training features is forbidden.

4. **Verify.** The top-ranked candidate is submitted to a deterministic verifier. The verifier is the grounding authority: exact checker, replay simulator, or human review depending on the domain. Its output is authoritative.
5. **Trace.** Every step is recorded in a machine-readable trace with stable action IDs. The trace becomes the evidence artifact that claims are bound to.

2.2. Feature Envelope and Leakage Policy

Every candidate action is encoded as a 96-dimensional feature vector. Each feature slot has a declared provenance: state observation (allowed), structural prior (allowed with audit), verifier-derived (forbidden during training). The leakage policy is mechanically enforced: the scorer's training loop receives only features whose provenance is compatible with the task's policy level. This means the model cannot peek at verifier outputs during training and then claim to be independent.

2.3. Rule Grammar and Certificates

Symbolic rules are governed by a domain-specific grammar. A rule consists of a kind (reject, transform, threshold), typed parameters, and a certificate. The certificate encodes the result of an exhaustive or algebraic check that the rule produces zero false rejections over its declared support. Rules are discovered by proposers—neural, structural, or random—and accepted only after certificate verification. Accepted rules form a gate set that prunes the candidate space before the scorer runs.

2.4. From Trace to Claim

The trace protocol uses stable action IDs so that every row in a trace file can be linked to a specific state, candidate set, model checkpoint, and verifier outcome. The paper's claim compiler reads trace artifacts and generates claim states: supported (evidence exists and meets thresholds), underpowered (evidence exists but sample count is insufficient), contradicted (evidence refutes the claim), or missing (no artifact found). Claims that are not supported by trace evidence appear in the limitations section, not the results.

3. Anchor: Arithmetic Sequence Filtering

The cleanest test case is next-prime gap prediction. Given a prime p below a configurable limit, the system must select the true gap to the next prime from 129 candidate even gaps plus an overflow action. The verifier is a segmented-sieve oracle that returns the exact next-prime gap and is never wrong inside the declared support. The gate grammar consists of modular rejection predicates: "reject candidate g if g > m and g % m == r" for modulus m in {3, 5, 7, …, 255} and residue r in {0, …, m-1}.

3.1. Learned Gate Discovery

A neural rule proposer enumerates candidate predicates. Each candidate is checked against the verifier over the training prime range. Accepted rules have zero false rejections: they never eliminate the true gap. The learned wheel discovers 51 rules (modular divisibility checks for primes 3 through 251) that collectively reduce the candidate set from 129 to exactly 1 for every prime in the support range.

The hand-coded wheel (small primes only, 17.7 modulus tests on average) achieves 77.6% top-1 and 100% top-10. The learned wheel achieves 100% top-1 and 100% top-10 with 58.8 modulus tests. The +0.02 pp pp improvement comes from discovering larger moduli that the hand wheel omits, while remaining fully certificate-verified.

3.2. What the Scorer Adds

After gating, the scorer ranks surviving candidates. With the learned wheel, only one candidate survives —the scorer is unnecessary. But the hand wheel leaves multiple survivors, and the scorer's ranking quality directly determines how many verifier calls are needed. The scorer reduces mean verifier steps

from 3.69 (ungated) to 1.07 (hand-wheel gated), a 71% reduction. The learned wheel eliminates verifier calls entirely for this task.

4. Holographic Rule Memory

The hypothesis library stores certified rules. As the rule set grows, flat-file storage and linear scanning become bottlenecks. We replace the flat file with a vector-symbolic memory substrate that stores rules as bindings on the unit hypersphere S^(d-1).

4.1. VSA Primitives

The memory operates on d-dimensional real vectors normalized to unit length. Three primitives, drawn from the vector symbolic architecture (VSA) literature [1], provide a Turing-complete computational substrate:

- **bind(k, v)** = normalize(ifft(fft(k) × fft(v))). Binds a key vector to a value vector via circular convolution. Implemented efficiently through FFT.
- **unbind(S, k)** = normalize(ifft(fft(S) × conj(fft(k)))). Recovers an approximation of v from a superposition S given key k.
- **bundle(a, b)** = normalize(a + b). Superposes vectors.

These are implemented in leOS-C [2], a compact C engine with pre-allocated FFT twiddle factors and zero malloc in the hot path.

4.2. Content-Addressable Rule Storage

To store a rule: M += bind(task_signature, rule_vector). The memory trace M is a single d-dimensional vector, regardless of how many rules are stored. To recall: rule_hat = unbind(M, query_task). The recalled vector is a weighted blend of all stored rules, with weight proportional to the cosine similarity between the query and each stored key.

4.3. Retrieval Fidelity

The theoretical retrieval fidelity is 1/sqrt(N) due to crosstalk between stored bindings in high-dimensional space [2]. Empirically, on a 5-class noisy recall task at d=512 with 200 stored examples, the holographic memory achieves 48.6% accuracy versus 20% chance. Adding holographic neurons (bind/unbind layers) before memory recall improves accuracy to 66% by learning a representation that separates classes in the VSA space.

4.4. Holographic Ranker

The holographic ranker replaces the neural scorer entirely with VSA memory lookup. Training is a single store operation per example. Inference is one unbind and dot-product sweep. On a 7-class task with 300 training examples at d=2048, the holographic ranker achieves 48.5% accuracy with zero trainable parameters, versus 16% for a gradient-trained MLP of 1,185 parameters. The holographic approach wins when sample counts are small and the representation is already meaningful; the neural approach wins when there is enough data to learn a task-specific embedding.

5. EML Neural Scoring

The neural scorer is a compact feedforward network trained by gradient descent. Its role is to rank candidates that survive the symbolic gate, providing the ordering pressure that minimizes verifier calls. We introduce a new computational primitive that improves the scorer's representational capacity without increasing depth or width.

5.1. The EML Operator

The EML operator is defined as:

$$\text{eml}(x, y) = \exp(x) - \ln(|y| + \varepsilon)$$

where the absolute value and small epsilon ensure the operator is defined for all real inputs. The operator is a universal primitive for continuous mathematics [3], analogous to the NAND gate for digital logic: repeated composition of EML operations can approximate any continuous function.

We integrate EML into the scorer in three forms, ordered by increasing architectural novelty.

5.2. Feature Enrichment

Before the first dense layer, pairwise EML features are appended to the envelope: $f_{96+i} = \text{eml}(f_{2i}, f_{2i+1})$. This adds 48 interaction terms, expanding the input to 144 dimensions. The enrichment is a fixed nonlinear transform; the subsequent dense layers learn which interactions matter.

On a 5-class nonlinear synthetic benchmark with overlapping clusters (800 train, 200 test, 64 features), a plain ReLU MLP (64->32->16) achieves 90% accuracy. With EML enrichment (96 features, same hidden widths), accuracy improves to 94.8%, outperforming both a wider ReLU (48->24, 94%) and a deeper ReLU (48->32->16, 91.5%) with fewer parameters than the deeper variant.

5.3. EML Activation

As a drop-in replacement for sigmoid or ReLU, the activation:

$$\text{eml}(x, 1 + e^x) = \exp(x) - \text{softplus}(x)$$

is smooth, non-negative, and has non-zero gradient everywhere. It approaches 0 from above for large negative inputs (like ReLU) and grows as exp(x) for positive inputs. With appropriate initialization (smaller variance, zero bias), it matches ReLU at 93% on the synthetic benchmark while eliminating the dead-neuron problem.

5.4. EML Pairwise Layer

The most architecturally novel form is a dedicated layer that applies EML to adjacent pre-activation pairs:

$$\text{output}_i = \text{eml}(z_{2i}, z_{2i+1})$$

where z_j = w_j · x + b_j are learned pre-activations. Each output neuron consumes two learned weight vectors, halving the effective width of the layer while doubling the per-output parameter budget. The forward pass, backpropagation through d/dx eml = exp(x) and d/dy eml = −1/y, and weight update are all handled inside the standard training loop. Pre-activation clipping at `[-5, 5]` prevents exponential overflow in deep stacks.

A single EML pairwise layer achieves 100% on the synthetic task. Two stacked EML layers (24 -> eml_pair(16) -> eml_pair(8) -> 3) achieve 94.5%, demonstrating that the universal-primitive property holds under gradient-based optimization.

5.5. When EML Helps

EML improves accuracy when features have meaningful pairwise continuous structure. On synthetic benchmarks with sinusoidal feature interactions, EML enrichment beats plain ReLU by 2.6 percentage points. On the promoted experiments—prime_gap (modular divisibility flags) and grid_path (binary cell occupancy)—EML does not help because the dominant features are binary or categorical, and the optimal ranking function is already well-approximated by linear combinations. EML is a tool for continuous structure, not a universal replacement.

6. Claim Governance

Claims are not prose. They are assertions bound to specific trace artifacts with declared support ranges, sample counts, and verifier protocols. The claim compiler reads the paper, extracts claim references, cross-references them against generated evidence files, and assigns each claim a status.

```
(
  supported: 14,
  contradicted: 1,
  underpowered: 5,
  missing: 0,
  stale: 0,
)
```

Claims that are supported appear in the results sections above. Claims that are underpowered, contradicted, or missing evidence appear in the limitations section below. This section is generated, not hand-written, and it updates automatically when new experiments produce evidence.

7. Limitations

```
(
  (
    claim_id: "C4.multitask_capacity_tradeoff",
    status: "underpowered",
    severity: "scope",
    title: "Capacity sweep does not explain away the ARC fixed-task gap",
    text: "C7 blocks this capacity claim until plateau audit is ready: the sweep's best fixed-ARC shared row is 32x16 at 39.20% and delayed_claim deltas are +0.10pp to +0.43pp, but only 20/72 plateau rows and 0/12 domain/variant verdicts are audit-ready.",
    roadmap_items: (),
    residual: (),
  ),
  (
    claim_id: "C4.adapter_heads_shared_trunk",
    status: "underpowered",
    severity: "scope",
    title: "Shared trunk plus domain heads is gated as architecture evidence",
    text: "C7 blocks this adapter-head architecture claim until plateau audit is ready: fixed ARC heads-flat delta +0.00pp; delayed_claim heads-flat +0.00pp; grid_path heads-flat +0.23pp; ARC OOD heads-single +5.80pp top-1 and -1.2020 NLL, with only 20/72 plateau rows and 0/12 domain/variant verdicts audit-ready.",
    roadmap_items: (),
    residual: (),
  ),
  (
    claim_id: "C7.tiny_net_plateau_stop_rule",
    status: "underpowered",
    severity: "scope",
    title: "Tiny-net architecture claims are gated by plateau audits",
    text: "tiny-net plateau audit covers 6 capacities, 3 restarts, and 8 checkpoints through 128 epochs across 12 domain/variant verdicts; 20/72 required rows meet the plateau threshold and 0/12 verdicts are audit-ready.",
    roadmap_items: (),
    residual: (),
  ),
```

```
  (
    claim_id: "C8.trace_determinism_audit",
    status: "contradicted",
    severity: "scope",
    title: "Trace determinism audit catches replay-sensitive claim drift",
   text: "the trace audit checks whether replay meaning survives numeric/runtime changes:
2 traces and 2000 rows, 0 invalid rows, 0 rank-sensitive rows, 0 logged mismatches, same
semantic hash: no.",
    roadmap_items: (),
    residual: (),
  ),
  (
    claim_id: "C5.arc_verified_self_play",
    status: "underpowered",
    severity: "scope",
    title: "ARC verified self-play is a fixed-budget search-efficiency claim",
      text: "ARC verified self-play is registered but underpowered: missing build/
float/arc_verified_self_play/summary.jsonl, so no public replay, fixed-budget ablation,
leakage audit, or zero-family opening can be credited.",
    roadmap_items: (),
    residual: (),
  ),
  (
    claim_id: "C9.signal_mining_replay_options",
    status: "underpowered",
    severity: "scope",
     title: "Signal mining option-ranker evaluates bounded policy hypotheses through
replay",
      text: "Signal mining claim registered as underpowered pending summary artifact
generation.",
    roadmap_items: (),
    residual: (),
  ),
)
```

The promoted evidence covers three task families (prime_gap, grid_path, delayed_claim) with exact-check and replay verifiers. The sandbox tasks (ARC, program synthesis, haiku, signal) are included as experimental proving grounds but do not yet meet the evidence thresholds for promoted claims.

The holographic memory substrate is demonstrated at dimensions up to d=2048 with hundreds of stored rules. Scaling to millions of rules in production requires dimension scaling or hierarchical memory structures that are not yet implemented.

The EML scorer improvements are demonstrated on synthetic benchmarks and the promoted experiments. The universal-primitive property is validated through stacked layer composition but not yet through formal function approximation bounds.

8. Conclusion

We presented an integrated architecture for verifier-grounded neural search. Three components compose: a certified gate framework that separates validity from preference through machine-checked rules, a holographic memory substrate that stores rules as fixed-dimension vector bindings with 1/sqrt(N) retrieval fidelity, and an EML-augmented neural scorer that improves ranking through a universal continuous primitive.

The integration is honest about what each component contributes and where it fails. EML does not help binary-feature domains. Holographic memory does not replace gradient-trained scorers on large datasets. Certified gates do not claim to solve problems outside their declared verifier support. But together they form a disciplined pipeline: enumerate, check, rank, trace, claim—with every assertion bound to machine-readable evidence.

Bibliography

[1] D. Kleyko *et al.*, "Vector Symbolic Architectures as a Computing Framework for Nanoscale Hardware and Emerging Applications," *Proceedings of the IEEE*, vol. 110, no. 10, pp. 1538–1571, 2022, doi: 10.1109/JPROC.2022.3209104.

[2] rati, "leOS-C – Holographic Engine in C." [Online]. Available: https://github.com/ratimics/leos-c

[3] Anonymous, "The EML Operator: A Universal Primitive for Continuous Mathematics." 2026.